

POSTGRESQL 18

Replication & High Availability

Native Replication • Patroni • repmgr • HAProxy • Keepalived • PgBouncer • Kubernetes HA

Document	Details
Audience	Senior PostgreSQL DBA / Cloud DBA / Database Engineer / Architect
Baseline	PostgreSQL 18.6 documentation and current vendor documentation
Validation date	9 September 2026
Focus	Replication, failover, HA orchestration, routing, fencing, monitoring and recovery

Core architecture principle: PostgreSQL provides the replication engine. HA orchestration, leader election, client routing, VIP management and connection pooling are separate layers. A production HA design combines the right components rather than expecting one tool to do everything.

Contents

1. Scope and HA Design Principles
2. Native PostgreSQL HA and Replication Landscape
3. Physical Streaming Replication
4. Asynchronous vs Synchronous Replication
5. Hot Standby and Read Scaling
6. Replication Slots
7. Cascading Replication
8. WAL Archiving and Log Shipping
9. Failover, Promotion, Timelines and Fencing
10. pg_rewind and Rejoining an Old Primary
11. Native Logical Replication
12. Logical Replication Failover
13. Monitoring Native Replication
14. Patroni
15. Distributed Configuration Store: etcd / Consul / ZooKeeper
16. HAProxy
17. Keepalived
18. PgBouncer
19. repmgr
20. EDB Failover Manager
21. pg_auto_failover
22. Pgpool-II
23. CloudNativePG and Kubernetes Operators
24. Distributed / Multi-Writer Options
25. Managed Cloud HA
26. Enterprise Comparison Matrix
27. Recommended Enterprise Architectures
28. RPO/RTO and Synchronous Design
29. Failure Scenarios and Runbooks
30. Security and Fencing
31. Production Monitoring Checklist
32. Interview Questions
33. Recommended Learning Roadmap
34. Official References

1. Scope and HA Design Principles

- High Availability (HA) reduces service interruption when a database node or infrastructure component fails.
- Replication copies changes; it is not a replacement for backup. Logical mistakes, corruption and destructive operations can propagate to replicas.
- RPO defines acceptable data loss. RTO defines acceptable recovery time.
- Automatic failover must include split-brain prevention. PostgreSQL documentation explicitly warns that an old primary must not return as an active primary after failover.
- Client routing must follow the current writable primary. A healthy database cluster is not useful if applications still connect to the failed node.
- Test planned switchover and unplanned failover regularly, including rejoin of the old primary.

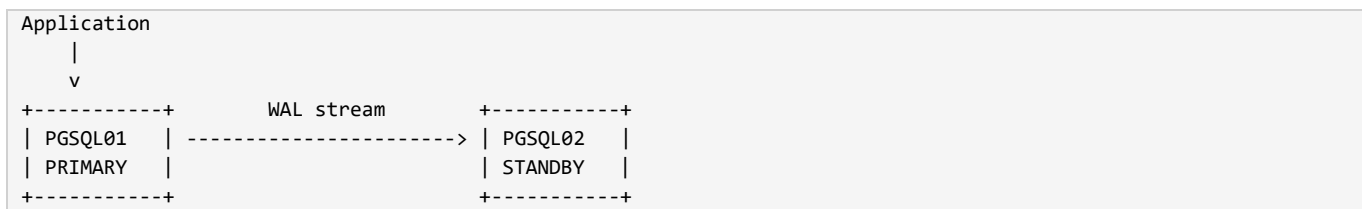
Important: PostgreSQL core provides strong replication and promotion primitives, but it does not by itself provide a complete enterprise cluster manager with leader election, fencing and client endpoint management.

2. Native PostgreSQL HA and Replication Landscape

Native capability	Purpose	Enterprise importance
Physical streaming replication	Continuously stream WAL from primary to standby	Critical
Hot standby	Read-only queries while standby applies WAL	Critical
Asynchronous replication	Low write latency with possible failover data loss	Critical
Synchronous replication	Wait for selected standby acknowledgement	Critical
Replication slots	Retain WAL required by physical/logical consumers	Critical
Cascading replication	Standby streams WAL to downstream standby	High
WAL archiving/log shipping	Archive/replay WAL for DR and standby recovery	Critical
Failover/promotion	<code>pg_ctl promote</code> / <code>pg_promote()</code>	Critical
<code>pg_rewind</code>	Resynchronize diverged old primary with new timeline	Critical
Logical replication	Publication/subscription, table-level replication	Critical
Logical failover slots	Allow logical subscriptions to continue after publisher failover	High
<code>pg_receivewal</code>	Stream WAL to external location	High

3. Physical Streaming Replication

Streaming replication sends WAL records from a primary server to one or more standby servers over PostgreSQL's replication protocol.



Primary preparation

```
# postgresql.conf
wal_level = replica
max_wal_senders = 10
```

```
max_replication_slots = 10
hot_standby = on

# pg_hba.conf - restrict to the real standby subnet in production
host replication replicator 10.10.10.0/24 scram-sha-256
```

```
CREATE ROLE replicator
WITH LOGIN REPLICATION
PASSWORD 'Use-A-Secret-Manager-In-Production';
```

Bootstrap a standby

```
pg_basebackup -h pgsq101 -U replicator -D /var/lib/postgresql/18/main -Fp -Xs -P -R
```

What -R does: `pg_basebackup -R` writes standby connection settings and creates `standby.signal` so the copied cluster starts as a standby.

4. Asynchronous vs Synchronous Replication

Mode	Commit behavior	RPO impact	Latency	Use case
Asynchronous	Primary does not wait for standby confirmation	Some committed transactions can be lost on failover	Lower	General HA, cross-region DR
Synchronous	Commit waits for configured synchronous acknowledgement	Can greatly reduce/avoid acknowledged transaction loss according to sync level	Higher	Critical transactional systems
Quorum synchronous	Wait for ANY N standbys	Strong durability with flexible quorum	Higher	3+ node HA

```
# Priority-based synchronous replication
synchronous_standby_names = 'FIRST 1 (pgsql02, pgsql03)'

# Quorum-based synchronous replication
synchronous_standby_names = 'ANY 1 (pgsql02, pgsql03)'
```

Design caution: Synchronous replication can block or slow commits when required synchronous standbys are unavailable. Design availability and durability together; do not enable it blindly across high-latency links.

5. Hot Standby and Read Scaling

A hot standby remains in recovery while accepting read-only queries. It can offload reporting/read workloads, but data can be slightly behind the primary and recovery conflicts can cancel long-running queries.

```
SELECT pg_is_in_recovery();

-- On standby, approximate replay delay
SELECT now() - pg_last_xact_replay_timestamp() AS replay_delay;
```

6. Replication Slots

Replication slots prevent PostgreSQL from removing WAL that a consumer still needs. They are useful but must be monitored because an inactive slot can retain large amounts of WAL.

```
SELECT * FROM pg_create_physical_replication_slot('pgsql02_slot');

SELECT slot_name, slot_type, active, restart_lsn,
       wal_status, safe_wal_size
FROM pg_replication_slots;
```

Production risk: Monitor inactive slots, retained WAL and max_slot_wal_keep_size. A forgotten slot can fill pg_wal and cause an outage.

7. Cascading Replication

```
PGSQL01 PRIMARY
  |
  v
PGSQL02 STANDBY
  |
  +-----> PGSQL03 STANDBY
  |
  +-----> PGSQL04 STANDBY
```

Cascading reduces the number of replication streams directly attached to the primary and can be useful for remote sites or large read-replica estates.

8. WAL Archiving and Log Shipping

WAL archiving can complement streaming replication. A standby can restore archived WAL when streaming is interrupted, and the archive also supports PITR.

```
archive_mode = on
archive_command = 'pgbackrest --stanza=pg18 archive-push %p'

# Standby recovery
restore_command = 'pgbackrest --stanza=pg18 archive-get %f "%p"'
```

9. Failover, Promotion, Timelines and Fencing

Promotion turns a standby into a writable primary. PostgreSQL supports promotion using `pg_ctl promote` or `pg_promote()`.

```
# OS command
pg_ctl -D /var/lib/postgresql/18/main promote

# SQL function
SELECT pg_promote();
```

```
Before failure:
PGSQL01 PRIMARY ----WAL----> PGSQL02 STANDBY
```

```
After promotion:
PGSQL01 OLD PRIMARY    X
PGSQL02 NEW PRIMARY    <--- applications
```

Required:

1. Fence/isolate PGSQL01.
2. Promote PGSQL02.
3. Route applications to PGSQL02.
4. Rewind/rebuild PGSQL01.
5. Rejoin PGSQL01 as standby.

Split-brain prevention: PostgreSQL documentation explicitly requires a mechanism to ensure the old primary knows it is no longer primary. Fencing/STONITH is a core HA requirement, not an optional cosmetic feature.

10. pg_rewind and Rejoining an Old Primary

`pg_rewind` can synchronize a PostgreSQL cluster that diverged from another copy of the same cluster after timeline divergence, often making old-primary rejoin much faster than taking a new full base backup.

```
# Stop old primary first.
pg_rewind --target-pgdata=/var/lib/postgresql/18/main --source-server="host=pgsql02 dbname=postgres
user=postgres"

# Configure it to follow the new primary, create standby.signal, then start.
touch /var/lib/postgresql/18/main/standby.signal
```

Prerequisite reminder: `pg_rewind` requires appropriate page checksums or `wal_log_hints` history and sufficient WAL/source access according to the PostgreSQL documentation. Validate prerequisites before relying on it in a runbook.

11. Native Logical Replication

Logical replication uses publications and subscriptions and can replicate selected tables. It is especially useful for migrations, cross-major-version upgrades, selective distribution and consolidation.

```
-- Publisher
CREATE PUBLICATION sales_pub
FOR TABLE public.customer, public.orders;

-- Subscriber
CREATE SUBSCRIPTION sales_sub
CONNECTION 'host=pgsql01 dbname=sales user=repuser password=REDACTED'
PUBLICATION sales_pub;
```

- Logical replication is table/database scoped rather than a physical byte-for-byte cluster copy.
- DDL is not automatically replicated as a general-purpose schema synchronization mechanism.
- Sequence state needs separate planning.
- Conflicts can occur when subscriber-side writes overlap replicated data.

12. Logical Replication Failover

PostgreSQL 18 supports synchronization of logical failover slots to a physical standby. This helps logical subscribers resume from the promoted publisher after failover when the configuration is designed correctly.

```
# Relevant PostgreSQL 18 server setting
sync_replication_slots = on

-- Monitor failover-ready logical slots as part of planned switchover.
SELECT slot_name, slot_type, failover, synced, active
FROM pg_replication_slots;
```

13. Monitoring Native Replication

```
-- Primary: connected physical standbys
SELECT pid, application_name, client_addr, state, sync_state,
       sent_lsn, write_lsn, flush_lsn, replay_lsn,
       write_lag, flush_lag, replay_lag
FROM pg_stat_replication;

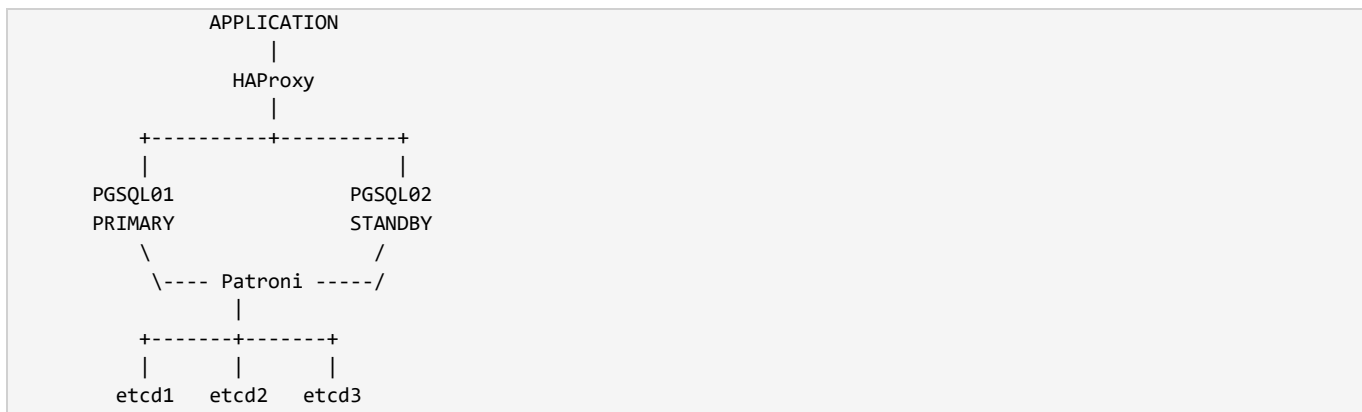
-- Standby: WAL receiver
SELECT status, sender_host, sender_port,
       written_lsn, flushed_lsn, latest_end_lsn,
       latest_end_time
FROM pg_stat_wal_receiver;

-- Slots
SELECT slot_name, slot_type, active, restart_lsn,
       confirmed_flush_lsn, wal_status
FROM pg_replication_slots;
```

14. Patroni

Patroni is a widely used open-source HA orchestrator for PostgreSQL. It uses PostgreSQL streaming replication and coordinates leader election and failover through a distributed configuration store (DCS).

Patroni capability	Purpose
Leader election	Determines which PostgreSQL node is primary
Automatic failover	Promotes an eligible healthy replica
Switchover	Controlled planned role change
Replication modes	Async and synchronous modes
REST API	Health/status endpoints used by routing layers
DCS integration	etcd, Consul, ZooKeeper and Kubernetes options
Rejoin	Can help orchestrate replica rebuild/rewind workflows
Configuration	Dynamic cluster configuration and PostgreSQL settings



Patroni is not replication: Patroni orchestrates PostgreSQL HA. PostgreSQL streaming replication remains the replication mechanism underneath.

15. DCS: etcd / Consul / ZooKeeper

A Patroni DCS stores cluster coordination state and participates in leader-election decisions. Use a fault-tolerant odd-sized DCS cluster such as three or five members.

- Keep DCS nodes in separate failure domains where possible.
- Do not place all DCS members on one hypervisor or one availability zone.
- Protect DCS networking and authentication.
- Monitor quorum, latency and disk health.
- Loss of DCS quorum affects HA control-plane decisions even if PostgreSQL processes remain running.

16. HAProxy

HAProxy provides a stable application endpoint and routes traffic based on health checks, commonly Patroni REST API checks.

```

Application
  |
  v
HAProxy :5432 ----> current PRIMARY
HAProxy :5433 ----> healthy REPLICAS

```

Layer responsibility: HAProxy routes connections. It does not decide PostgreSQL data consistency or perform database promotion by itself.

17. Keepalived

Keepalived can provide a floating Virtual IP using VRRP so two HAProxy nodes do not create a frontend single point of failure.

```

Applications
  |
  v
VIP 10.10.10.100
  |
+---+-----+
|               |
| HAProxy01     | HAProxy02
| Keepalived    | Keepalived
|               |
+----- PG HA -----+

```

18. PgBouncer

PgBouncer is a lightweight PostgreSQL connection pooler. It is not a replication or failover engine, but it is commonly used in HA designs to reduce connection overhead and protect PostgreSQL from connection storms.

Mode	Behavior	HA consideration
Session	Backend held for client session	Most compatible; less pooling efficiency
Transaction	Backend returned after transaction	Very efficient; session-state features need care
Statement	Backend returned after statement	Most restrictive; not suitable for many applications

19. repmgr

repmgr is a PostgreSQL replication-management suite that assists with standby cloning, monitoring, switchover, promotion, failover and node rejoin.

- repmgr CLI for administration
- repmgrd daemon for monitoring/failover
- Native streaming replication underneath
- Good alternative where teams prefer a simpler replication-focused management model

20. EDB Failover Manager

EDB Failover Manager is a commercial/enterprise HA manager from EDB for PostgreSQL and EDB Postgres environments. It provides monitoring and automated failover capabilities and is most relevant where EDB support and tooling are part of the enterprise standard.

21. pg_auto_failover

pg_auto_failover is an automated PostgreSQL failover project built around a monitor plus PostgreSQL nodes. Treat it as a valid architecture option, but evaluate current project maintenance, support requirements and organizational standards before selecting it for a new enterprise platform.

22. Pgpool-II

Pgpool-II is middleware between clients and PostgreSQL and can provide connection pooling, load balancing, health checking and failover-related functions. It is not a replacement for understanding PostgreSQL replication.

Architecture caution: Do not combine multiple independent failover managers without a clear ownership model. Two components trying to promote/fail over nodes can create dangerous race conditions.

23. CloudNativePG and Kubernetes Operators

For Kubernetes, operators such as CloudNativePG manage PostgreSQL lifecycle, replication, services, failover, backups and cluster reconciliation using Kubernetes-native control mechanisms.

Platform	Role
CloudNativePG	PostgreSQL Kubernetes operator with HA, failover, services and backup integration
StackGres	PostgreSQL Kubernetes platform/operator with operational tooling
Zalando Postgres Operator	Kubernetes PostgreSQL cluster lifecycle and HA automation

Kubernetes principle: Use an operator that owns PostgreSQL lifecycle and failover. Avoid manually layering a second independent HA manager unless the operator explicitly supports that design.

24. Distributed / Multi-Writer Options

Core PostgreSQL does not provide synchronous multi-master replication as a built-in HA mode. Multi-writer/distributed requirements normally require external technology and conflict-resolution design.

Technology	Category	Use
EDB Postgres Distributed (PGD)	Commercial distributed PostgreSQL	Multi-writer/distributed HA and conflict management
Bucardo	Third-party logical replication	Asynchronous multi-master / multi-source patterns
pglogical	Logical replication extension/ecosystem	Advanced logical replication patterns; evaluate against modern core logical replication

25. Managed Cloud HA

Platform	Typical HA model	DBA focus
Amazon RDS for PostgreSQL	Multi-AZ managed standby/failover options	Parameter design, endpoints, failover testing, backups
Amazon Aurora PostgreSQL	Distributed storage + writer/readers	Cluster endpoint, reader endpoint, failover priority
Azure Database for PostgreSQL	Managed HA options	Zone design, failover behavior, connection retry
Google Cloud SQL for PostgreSQL	Regional HA options	Failover, maintenance, connectivity, backups

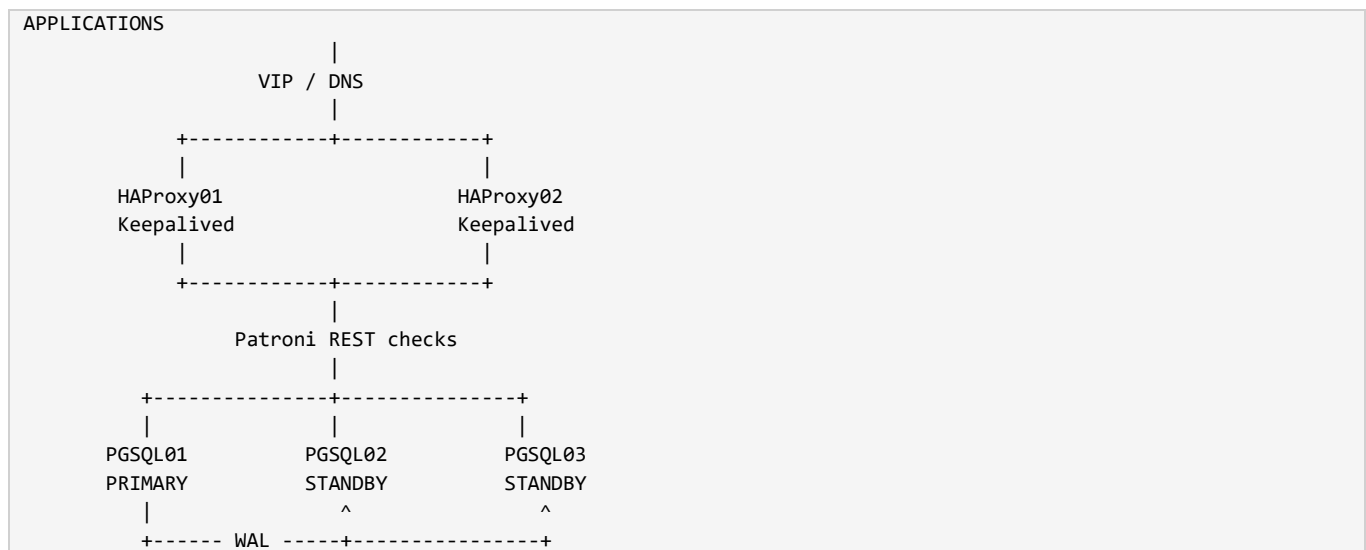
Managed cloud HA removes some infrastructure work, but application retry logic, connection endpoints, RPO/RTO validation, backup strategy and failover testing remain DBA/architecture responsibilities.

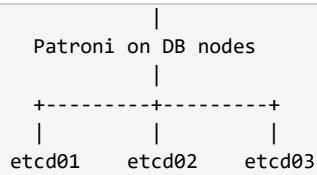
26. Enterprise Comparison Matrix

Tool / Feature	Layer	Auto failover	Routing	Pooling	Best fit
Native streaming replication	Replication	No full orchestration	No	No	Foundation for physical HA
Patroni	HA orchestration	Yes	Via integrations	No	General enterprise self-managed HA
repmgr	Replication/HA management	Yes with repmgrd	External	No	Replication-focused HA
EDB Failover Manager	Commercial HA	Yes	Integration dependent	No	EDB-supported enterprise
HAProxy	Routing	No	Yes	No	Primary/replica endpoints
Keepalived	VIP	No	VIP only	No	HAProxy/frontend redundancy
PgBouncer	Connection pool	No	Limited/indirect	Yes	Connection efficiency
Pgpool-II	Middleware	Failover-related features	Yes	Yes	Middleware-centric architecture
CloudNativePG	Kubernetes operator	Yes	K8s Services	Integrated patterns	Kubernetes PostgreSQL

27. Recommended Enterprise Architectures

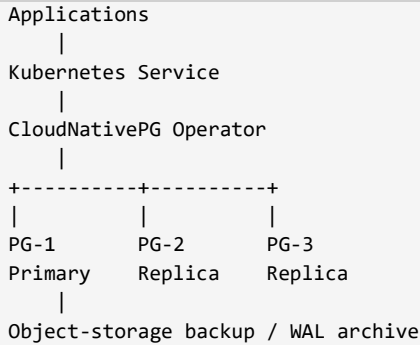
A. Linux / VM / EC2 - preferred reference architecture



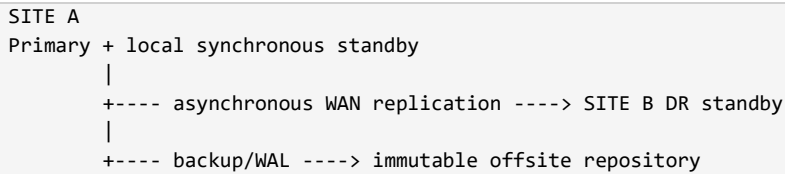


Backup/DR layer: pgBackRest + WAL archive + offsite repository

B. Kubernetes



C. Multi-site DR



Why this pattern: Use synchronous replication inside a low-latency failure domain when the business requires stronger durability, and asynchronous replication across long-distance WAN links to avoid excessive commit latency.

28. RPO/RTO and Synchronous Design

Requirement	Possible design	Trade-off
RPO minutes	Async streaming + WAL archive	Low latency, small loss possible
RPO near zero within site	Synchronous standby	Higher commit latency / availability trade-off
Low RTO	Automated HA + prebuilt standby + routing layer	More components and testing
Regional disaster recovery	Async remote standby + offsite backup	WAN lag determines potential loss
Read scaling	Hot standbys + read endpoint	Replica lag and query conflicts

29. Failure Scenarios and Runbooks

Primary PostgreSQL process crashes

HA manager verifies failure, selects eligible standby, promotes it, routing layer moves writes, old node is fenced/rejoined.

Primary host loses network

Treat as a split-brain risk. Do not promote another node unless the old primary is safely fenced or the HA system can guarantee leadership ownership.

Synchronous standby fails

Depending on synchronous policy, commits may wait. HA manager can select another eligible synchronous standby or the operator can deliberately change durability policy.

Standby falls far behind

Investigate network, I/O, WAL generation, replay bottlenecks and slots. Do not promote a badly lagging replica unless the accepted RPO permits it.

Old primary returns

Keep it isolated. Use `pg_rewind` when eligible or rebuild from a fresh base backup, then rejoin as standby.

DCS loses quorum

PostgreSQL may still serve traffic depending on architecture, but Patroni cannot safely perform normal leader-election actions. Restore DCS quorum; avoid manual competing promotions.

HAProxy node fails

Keepalived/VIP or another load balancer should move client entry point to the surviving proxy.

Accidental DELETE/TRUNCATE

HA replication is not the recovery mechanism because the change replicates. Use backup/PITR or logical recovery strategy.

30. Security and Fencing

- Use SCRAM authentication and least-privilege replication roles.
- Restrict replication connections by network and `pg_hba.conf`.
- Protect Patroni REST API and DCS endpoints from untrusted networks.
- Use TLS where required for database, DCS and proxy communication.
- Design a tested fencing mechanism for old-primary isolation.
- Protect replication credentials with secret stores rather than plaintext scripts.
- Audit switchover/failover actions and configuration changes.
- Ensure backup/PITR remains independent of the HA cluster.

31. Production Monitoring Checklist

Area	Monitor
Primary/standby	pg_stat_replication state, sync_state, LSN positions
Standby	pg_stat_wal_receiver and replay timestamp
Lag	byte lag + time/replay lag; define workload-specific thresholds
Slots	active status, restart_lsn, retained WAL, wal_status
WAL	pg_wal growth, archive health, WAL generation rate
Patroni	leader, member health, timeline, pending restart, DCS connectivity
DCS	quorum, latency, disk, member health
HAProxy	backend health and current primary routing
PgBouncer	client/server connections, pool saturation, wait time
Failover readiness	regular controlled switchover and DR drills

32. Interview Questions

Q: Is PostgreSQL streaming replication synchronous by default?

A: No. Physical streaming replication is normally asynchronous unless synchronous replication is explicitly configured.

Q: Does Patroni replicate PostgreSQL data?

A: No. PostgreSQL streaming replication moves the data/WAL. Patroni orchestrates HA, leader election and failover.

Q: Why is fencing required?

A: To prevent the old primary and new primary from both accepting writes after a failure, which can cause split brain and data loss.

Q: What is pg_rewind used for?

A: To synchronize a diverged old primary or other cluster copy with the new primary timeline when prerequisites are satisfied.

Q: Can a hot standby accept writes?

A: No. It can serve read-only queries while recovery is active. It becomes writable after promotion.

Q: What is the risk of replication slots?

A: Inactive or lagging slots can retain WAL and consume disk space.

Q: FIRST vs ANY synchronous_standby_names?

A: FIRST chooses synchronous standbys by priority; ANY uses quorum semantics and waits for any required number of listed standbys.

Q: Is HA a backup?

A: No. Replication can reproduce user errors and corruption. Backup/PITR is a separate protection layer.

Q: Why use HAProxy with Patroni?

A: To provide applications a stable endpoint that routes connections to the node Patroni reports as the current primary or healthy replica.

Q: Why use Keepalived with HAProxy?

A: To avoid a single proxy endpoint by providing a movable VIP across multiple proxy hosts.

Q: Where does PgBouncer fit?

A: Connection pooling. It reduces backend connection overhead but is not the failover manager.

Q: When would logical replication be preferred?

A: Selective table replication, cross-major-version migration/upgrades, consolidation/distribution and heterogeneous platform movement.

33. Recommended Learning Roadmap

Priority	Topic	Depth
1	WAL, timelines, recovery fundamentals	Expert
2	Physical streaming replication	Expert
3	Async / sync / quorum replication	Expert
4	Hot standby, slots, cascading	Expert
5	Failover, fencing, pg_rewind	Expert
6	Logical replication + failover slots	Expert
7	Patroni	Expert
8	etcd/DCS	Strong
9	HAProxy + Keepalived	Strong
10	PgBouncer	Strong
11	repmgr	Strong
12	CloudNativePG / Kubernetes HA	Strong for cloud-native roles
13	EDB FFM / Pgpool-II / pg_auto_failover	Working knowledge
14	Distributed multi-writer systems	Architect-level as required

Recommended hands-on lab sequence: Streaming replication -> replication slots -> synchronous replication -> cascading -> planned switchover -> forced failover -> fencing -> pg_rewind/rejoin -> Patroni + 3-node etcd -> HAProxy -> Keepalived -> PgBouncer -> backup/PITR integration.

34. Official References

PostgreSQL 18 - High Availability, Load Balancing and Replication

<https://www.postgresql.org/docs/18/high-availability.html>

PostgreSQL 18 - Replication Configuration

<https://www.postgresql.org/docs/18/runtime-config-replication.html>

PostgreSQL 18 - Failover

<https://www.postgresql.org/docs/18/warm-standby-failover.html>

PostgreSQL 18 - Logical Replication

<https://www.postgresql.org/docs/18/logical-replication.html>

PostgreSQL 18 - Logical Replication Quick Setup

<https://www.postgresql.org/docs/18/logical-replication-quick-setup.html>

PostgreSQL 18 - pg_rewind

<https://www.postgresql.org/docs/18/app-pgrewind.html>

PostgreSQL 18 - pg_receivewal

<https://www.postgresql.org/docs/18/app-pgreceivewal.html>

Patroni Documentation

<https://patroni.readthedocs.io/en/latest/>

Patroni Replication Modes

https://patroni.readthedocs.io/en/latest/replication_modes.html

repmgr Documentation

<https://www.repmgr.org/docs/current/>

CloudNativePG Documentation

<https://cloudnative-pg.io/documentation/>

PgBouncer Documentation

<https://www.pgbouncer.org/usage.html>

Pgpool-II Documentation

<https://www.pgpool.net/docs/latest/en/html/>

HAProxy Documentation

<https://www.haproxy.com/documentation/haproxy-configuration-manual/latest/>

Keepalived Documentation

<https://www.keepalived.org/manpage.html>